Heart Disease Risk Prediction – End-to-End MLOps Pipeline

Course: AIMLCZG523 – MLOps

Assignment 1

Name: Sri Vishnu Vallabh Yathavakilla

Roll No. (BITS ID): 2025AE05312

Date: 19 October 2026

Code repository: https://github.com/Vishnuvallabh/heart-disease-mlops

Video: [video link]

1. Introduction

The goal of this project is to predict whether a patient has heart disease from 13 simple medical measurements, and to run that prediction as a reliable service. The focus is not only the model, but everything around it: getting the data, tracking experiments, testing, packaging, deploying and monitoring.

Dataset: UCI Heart Disease (Cleveland). It has 303 patients and 13 features such as age, sex, chest pain type, blood pressure, cholesterol and maximum heart rate. The original target column num goes from 0 (no disease) to 4 (severe disease).

Final result: a Logistic Regression model with 0.958 ROC-AUC on unseen test patients, served by a FastAPI service with 2 copies on Kubernetes, checked by a GitHub Actions pipeline on every push, and watched by Prometheus and Grafana.

2. Architecture

The project has three stages.

- Stage 1 – Laptop: download the data, clean it, explore it in a notebook, train two models, and record every run in MLflow. The best model is saved as one file.
- Stage 2 – GitHub Actions: on every push, the code is checked for style (lint), tested, the model is retrained, and the Docker image is built and tested. If any step fails, the next steps do not run.
- Stage 3 – Kubernetes (Docker Desktop): the API runs as 2 pods behind a LoadBalancer on localhost:8080. Prometheus collects metrics from both pods every 5 seconds and Grafana shows them on a dashboard.

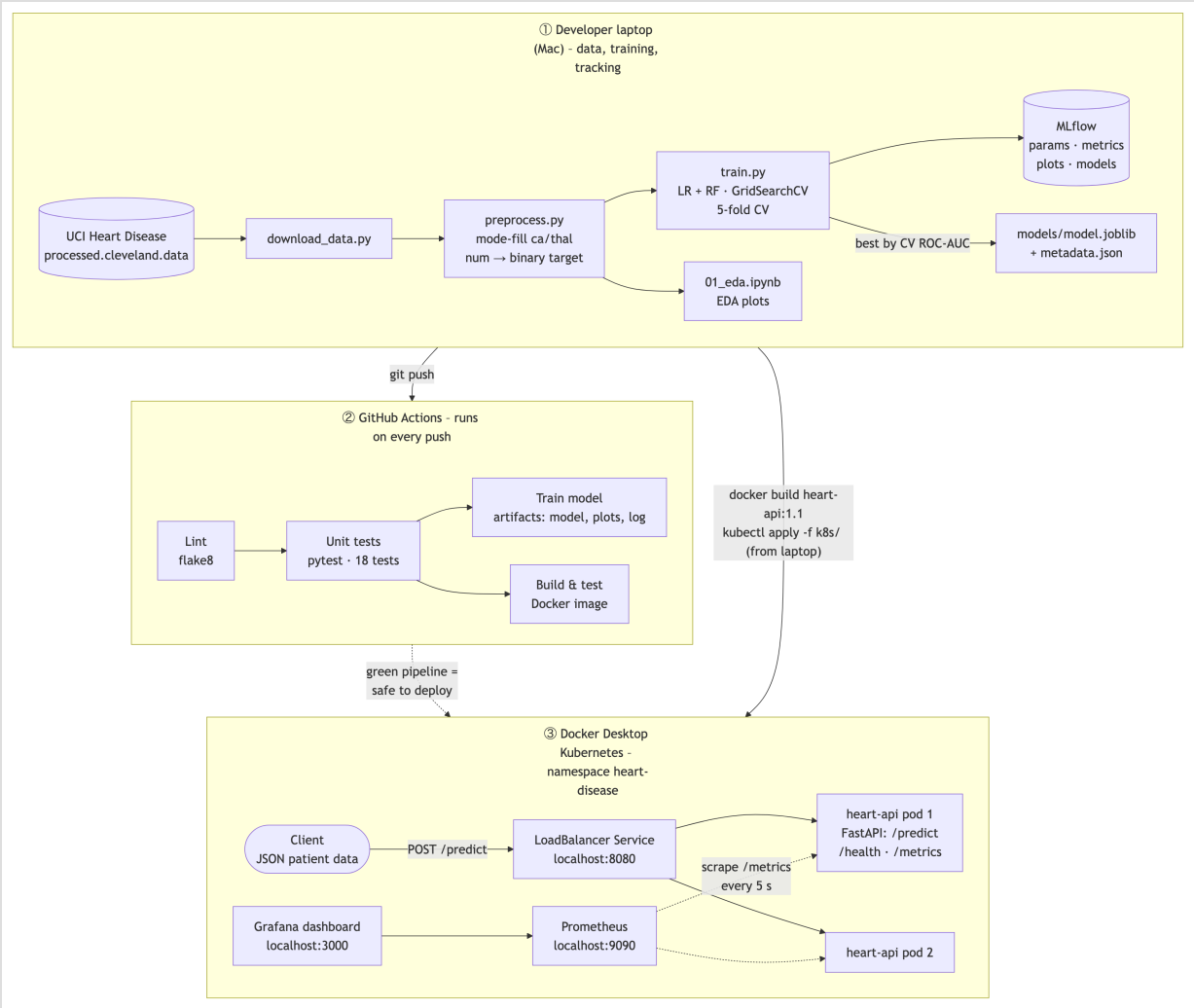


Figure 1 – Architecture: laptop → GitHub Actions → Kubernetes with monitoring.

Tools used: Python 3.12, pandas, scikit-learn, MLflow, pytest, flake8, GitHub Actions, FastAPI, Docker, Kubernetes, Prometheus and Grafana. All are free and run locally.

3. Data and exploratory analysis (EDA)

3.1 Getting and cleaning the data

- src/download_data.py downloads processed.cleveland.data from the UCI website, so anyone can get the same raw file.
- The raw file has no header row, so the 14 column names are added when loading. Missing values are written as '?', so they are read as missing.
- 6 of 303 rows (about 2%) had a missing value: 4 in ca and 2 in thal. They were filled with the most frequent value (the mode). Both columns are categories or small counts (ca = 0–3 vessels, thal = 3/6/7), so an average like 0.67 vessels would not make sense. Filling keeps all 303 patients.
- num (0–4) was turned into a yes/no target: 0 = no disease, 1 = disease (num 1–4).
- src/preprocess.py does all of this and saves data/processed/heart_clean.csv.

3.2 Class balance

There are 164 patients without disease (54%) and 139 with disease (46%). The classes are roughly balanced, so no balancing technique was needed. A model that always says 'no disease' would get only 54% accuracy – that is the baseline to beat.

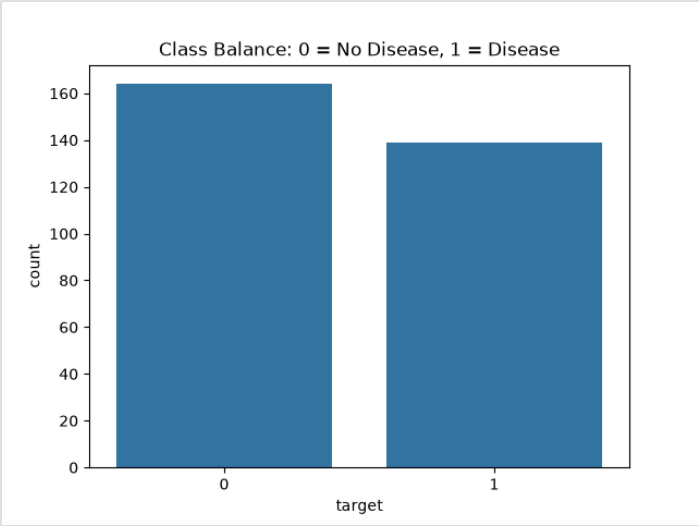


Figure 2 – Class balance: 164 without disease, 139 with disease.

3.3 Distributions

Most patients are 48–61 years old (median 56). Cholesterol is mostly 211–275 but has a few outliers up to 564. oldpeak has a large spike at 0 (99 patients). The features use very different scales (cholesterol in the hundreds, oldpeak from 0 to 6), which is why the numeric features are scaled before training.

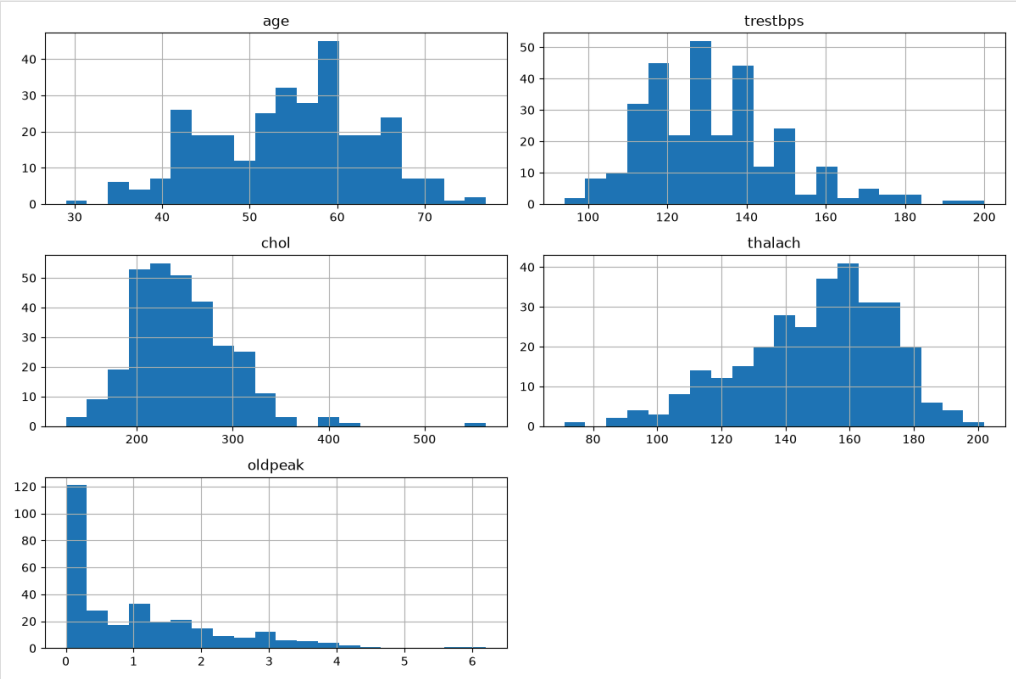


Figure 3 – Histograms of the numeric features.

3.4 Correlations

Looking at the target row of the heatmap: thal (0.52), ca (0.46), exang (0.43), oldpeak (0.42) and cp (0.41) have the strongest positive link with disease. thalach (maximum heart rate) is clearly negative (–0.42): patients who reach a higher heart rate during exercise are less likely to have disease. fbs (0.03) and cholesterol (0.09) are close to zero. Because cp, thal, restecg and slope are categories, their correlation values are only a rough guide.

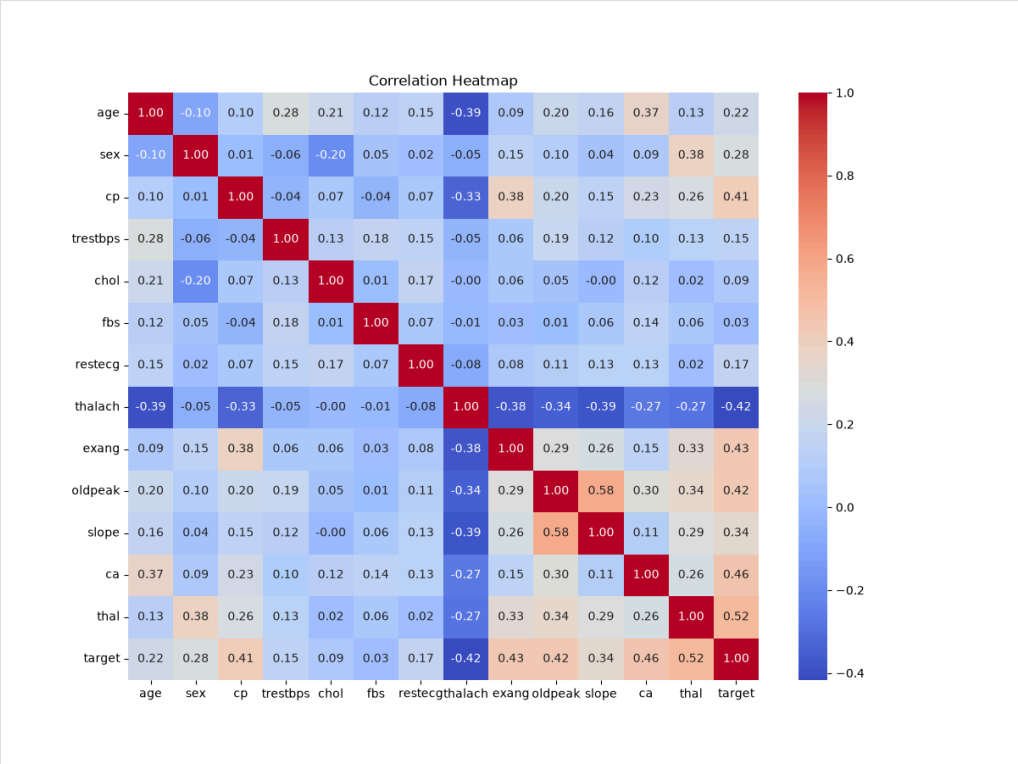


Figure 4 – Correlation heatmap.

4. Features and model development

4.1 Preparing the features

Group	Columns	Treatment
Numeric	age, trestbps, chol, thalach, oldpeak, ca	Scaled (StandardScaler)
Categorical	cp, restecg, slope, thal	One-hot encoded
Binary (0/1)	sex, fbs, exang	Left as they are

Scaling puts all numeric columns on the same scale so no column looks important just because its numbers are big. One-hot encoding turns category labels into yes/no columns so the model does not treat 'chest pain type 4' as twice 'type 2'. ca is a real count (0–3), so it is treated as numeric.

The scaler, the encoder and the model are joined into one scikit-learn Pipeline. This has two benefits: the scaler only learns from training data (no data leakage), and the exact same steps are applied when the API receives a new patient.

4.2 Training and tuning

- Split: 80% training (242 patients), 20% test (61 patients), stratified so both parts keep the same 54/46 ratio. random_state=42 makes it repeatable.
- Two models: Logistic Regression (simple and easy to explain) and Random Forest (many decision trees voting).
- Tuning with GridSearchCV and 5-fold stratified cross-validation on the training set. Logistic Regression: C = 0.01, 0.1, 1, 10. Random Forest: n_estimators = 100, 300 and max_depth = None, 5, 10.
- Metrics: accuracy, precision, recall and ROC-AUC. Recall matters most in medicine: a missed sick patient is worse than a false alarm.

4.3 Results

Model	Best settings	CV ROC-AUC	Accuracy	Precision	Recall	Test ROC-AUC
Logistic Regression (final)	C = 1	0.907	0.869	0.8125	0.929	0.958
Random Forest	300 trees, depth 10	0.895	0.885	0.818	0.964	0.947

Both models are good and very close – on 61 test patients the difference is one patient. Logistic Regression caught 26 of 28 sick patients with 6 false alarms; Random Forest caught 27 of 28, also with 6 false alarms. Random Forest's settings made almost no difference (all combinations scored within about 0.003 CV ROC-AUC).

The final model was chosen by CV ROC-AUC, not by test score, so the test set stays a fair, unseen check. Logistic Regression wins (0.907 vs 0.895), and it is also simpler to explain.

5. Experiment tracking with MLflow

Every training run is recorded in MLflow under the experiment 'heart-disease'. Each run of train.py creates 12 MLflow runs: one parent run per model, plus one child run for every setting tried (4 for Logistic Regression, 6 for Random Forest).

- Parameters: model type, number of CV folds, best settings.
- Metrics: CV ROC-AUC, test accuracy, precision, recall and ROC-AUC.
- Artifacts: confusion matrix and ROC curve plots, and the full trained pipeline.

The Compare view puts both models side by side, which made the final choice easy to see and to justify.

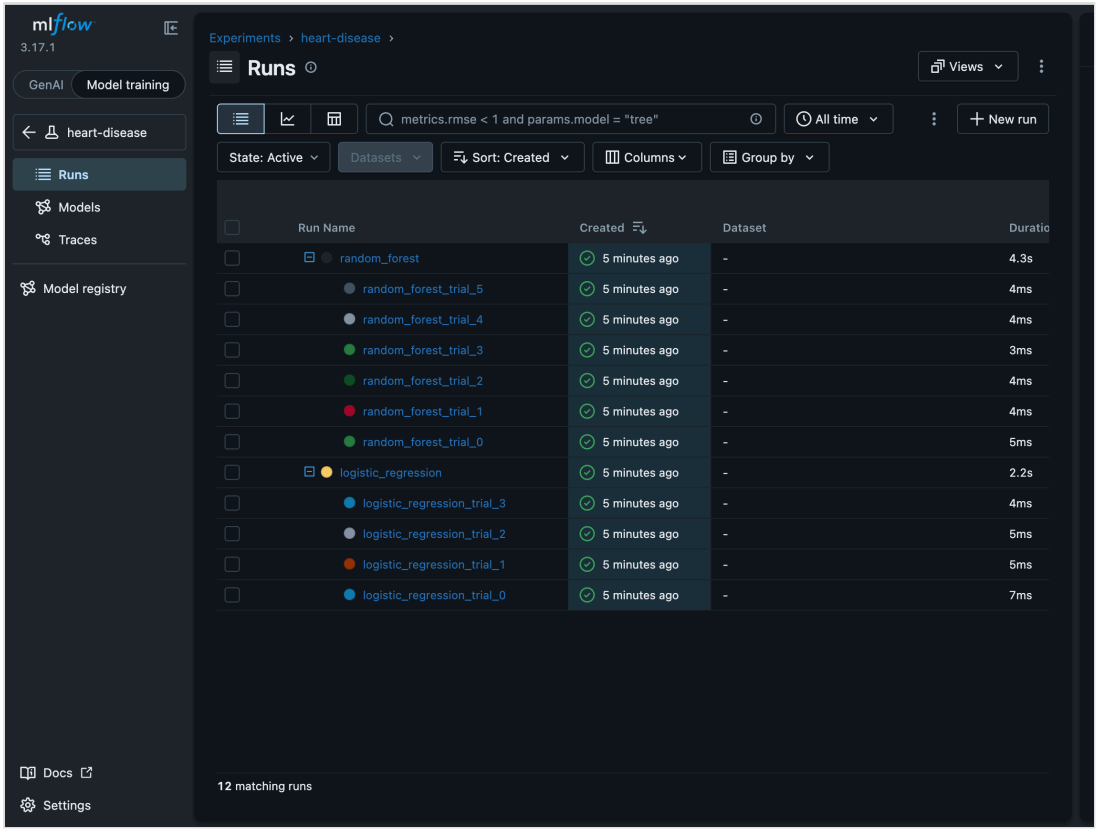


Figure 5 – MLflow: 2 parent runs and 10 child runs (one per setting tried).

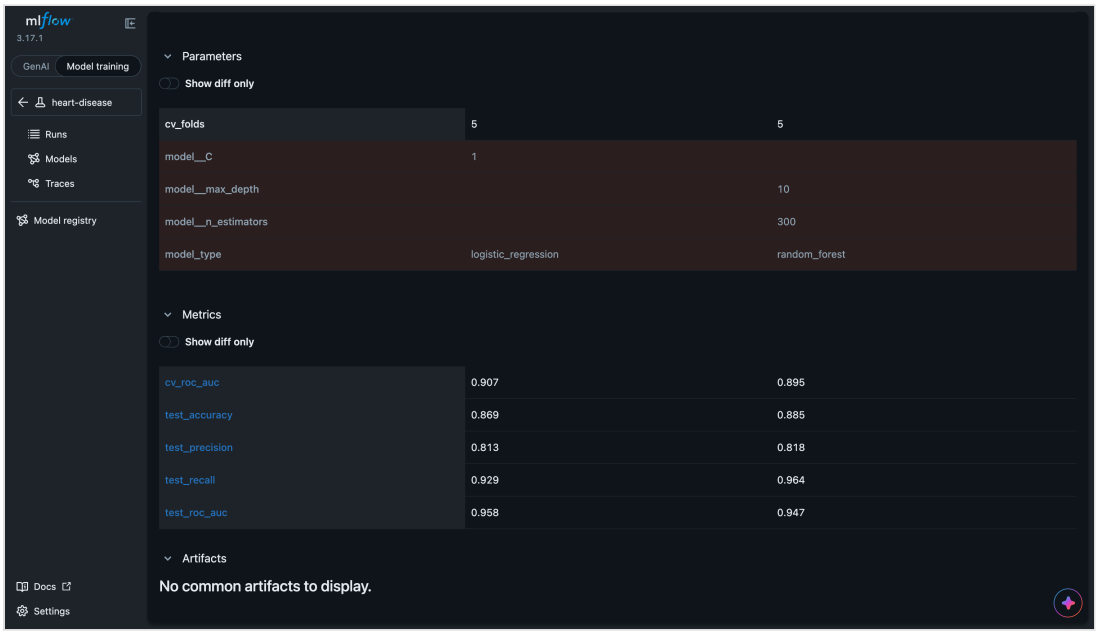


Figure 6 – MLflow Compare view: Logistic Regression vs Random Forest.



Figure 7 – Run artifacts: Random Forest confusion matrix (27 + 27 correct, 6 false alarms, 1 missed).

6. Model packaging and reproducibility

- models/model.joblib holds the whole pipeline (scaler + encoder + model), so new data gets exactly the same treatment as training data.
- models/metadata.json stores the model name, the feature order, the metrics and the scikit-learn version.
- requirements.txt pins the exact version of every library; requirements-api.txt holds only what the API needs.
- src/predict.py predicts one patient from a JSON file: the sample patient gives prediction 0 (no disease) with confidence 0.823.
- Reproducibility test: a brand-new environment built only from requirements.txt ran download → train → predict and gave identical numbers. A fresh clone from GitHub also passed all tests.



Figure 8 – Clean-setup test: fresh environment, same results, same prediction.

7. Testing and CI/CD

7.1 Unit tests

There are 18 pytest tests: 5 for data cleaning (missing values become empty, no gaps remain, target is 0/1, no rows are lost, input is not changed), 7 for the model code (scaling, one-hot encoding, valid probabilities, all metrics present, prediction format, unseen category does not crash), and 6 for the API (health, prediction, bad input rejected, metrics page, logging). The tests use small fake data, so they are fast, need no internet, and their correct answers are known in advance.

7.2 GitHub Actions pipeline

Job	What it does	Runs after
Lint	flake8 style check + Kubernetes manifest check	–
Unit tests	pytest -v, saves test-results.xml	Lint
Train model	download → preprocess → train → prediction check; saves model, plots, log	Unit tests
Build & test Docker image	builds the image, starts it, checks /health and /predict	Unit tests

The order is cheap checks first: there is no point spending minutes training a model if the code has a style error or a failing test. Every run saves its artifacts (test report, model, plots, training log) for download.

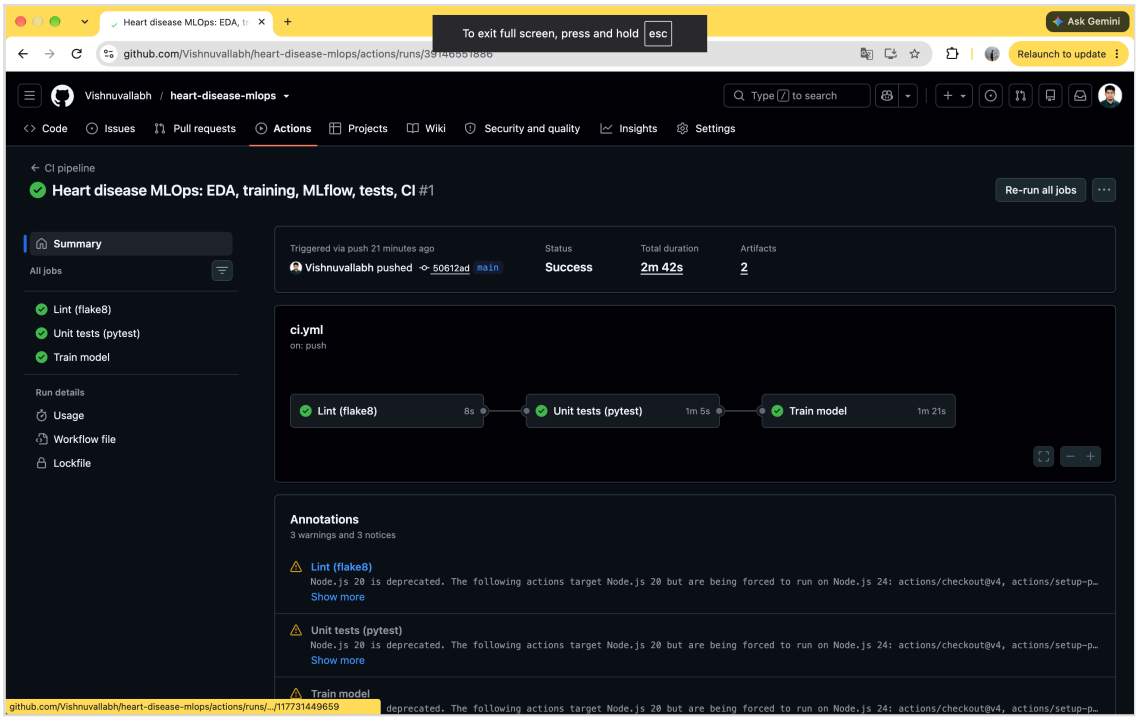


Figure 9 – Green pipeline run with the jobs chained in order (taken before the Docker job was added).

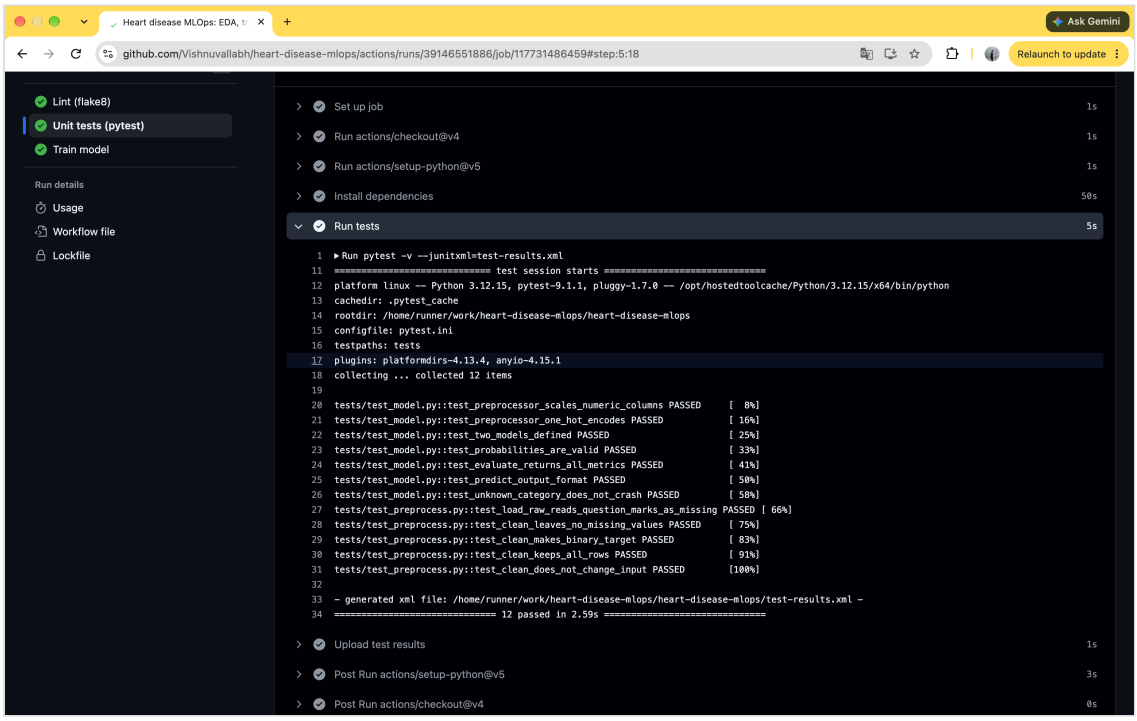


Figure 10 – Test log in GitHub Actions (12 tests at that stage; 18 after the API tests were added).

7.3 Proof that failures are caught

A test was broken on purpose and pushed. Lint passed, Unit tests failed, and Train model was skipped. The log named the exact test and the wrong values. The change was then reverted and the pipeline was green again.

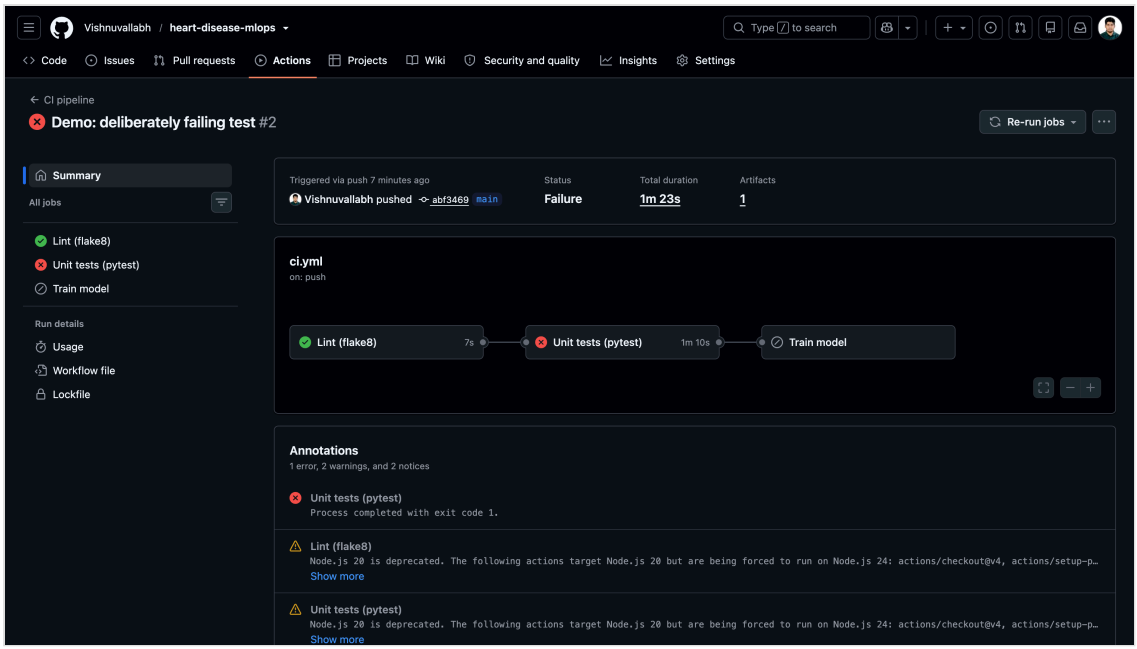


Figure 11 – Deliberately broken test: tests fail, training is skipped.

8. Containerization

The model is served by a FastAPI service (src/app.py) with these endpoints:

- POST /predict – takes patient JSON, returns prediction, label and confidence.
- GET /health – used by Docker and Kubernetes to check the service is alive.
- GET /metrics – numbers for Prometheus (added in the monitoring step).
- GET /docs – interactive page to try the API in a browser.

Every input field is checked (for example age must be 1–120 and thal must be 3, 6 or 7). Bad input gets a clear 422 error instead of a crash or a nonsense prediction.

The Dockerfile uses the small python:3.12-slim image, installs only the API libraries, copies only the code and the model, runs as a non-root user for safety, and has a health check. The container answered the sample patient with 0 / 0.823, a high-risk patient with 1 / 0.998, and age 500 with a 422 error. GitHub Actions repeats this test on every push.

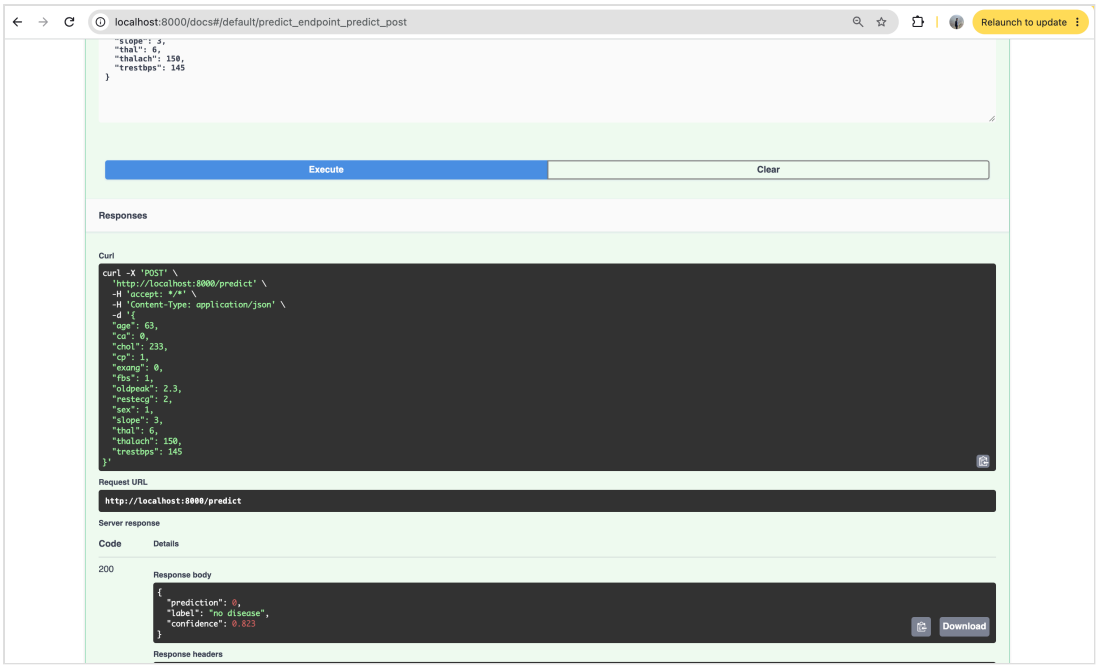


Figure 12 – /docs page: the sample patient returns no disease, confidence 0.823.



Figure 13 – Running container: healthy, correct predictions, and 422 for invalid input.

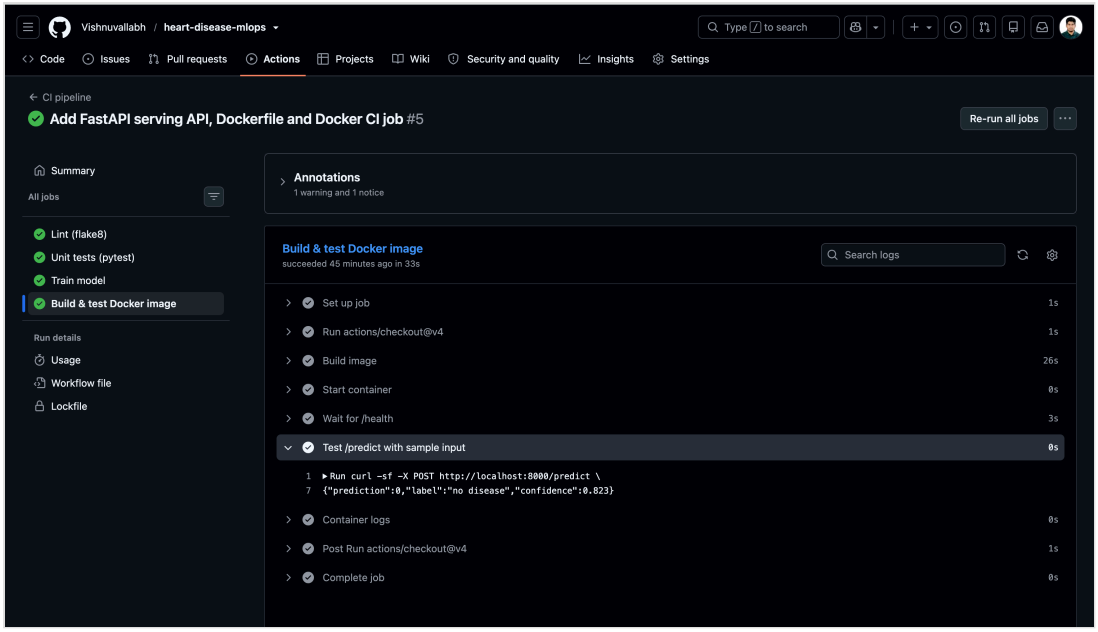


Figure 14 – The Docker job in GitHub Actions builds and tests the container.

9. Production deployment on Kubernetes

The API is deployed on Docker Desktop's built-in Kubernetes (kubeadm, v1.36.1). A local cluster costs nothing; the same manifests would work on a cloud cluster (GKE, EKS, AKS) after pushing the image to a registry.

- k8s/namespace.yaml – keeps all project objects together in 'heart-disease'.
- k8s/deployment.yaml – 2 replicas (one can fail while the other serves), CPU/memory limits, readiness and liveness checks on /health, and rolling updates that replace one pod at a time with no downtime.
- k8s/service.yaml – a LoadBalancer that exposes the API at localhost:8080.

Checks done: all three endpoints answered through the LoadBalancer; 20 requests were spread over both pods; deleting a pod made Kubernetes create a new one within seconds while /health kept answering (self-healing); and the deployment scaled to 3 pods and back with one command. Later, the update to image 1.1 (with monitoring) was rolled out pod by pod with no downtime.

```
srivishnuvallabhyathavakilla@Sris-MacBook-Pro MLOPS % clear
kubectl get all -n heart-disease
NAME                             READY   STATUS    RESTARTS   AGE
pod/heart-api-86bbbc8b98-fs5g7   1/1     Running   0          5m41s
pod/heart-api-86bbbc8b98-wllfj   1/1     Running   0          6m1s

NAME                 TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/heart-api    LoadBalancer   10.111.251.187   localhost     8080:32742/TCP   6m1s

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-api    2/2     2            2           6m1s

NAME                                   DESIRED   CURRENT   READY   AGE
replicaset.apps/heart-api-86bbbc8b98   2         2         2       6m1s
srivishnuvallabhyathavakilla@Sris-MacBook-Pro MLOPS %
```

Figure 15 – 2 pods running and the LoadBalancer service on localhost:8080.



Figure 16 – Endpoints verified through Kubernetes.

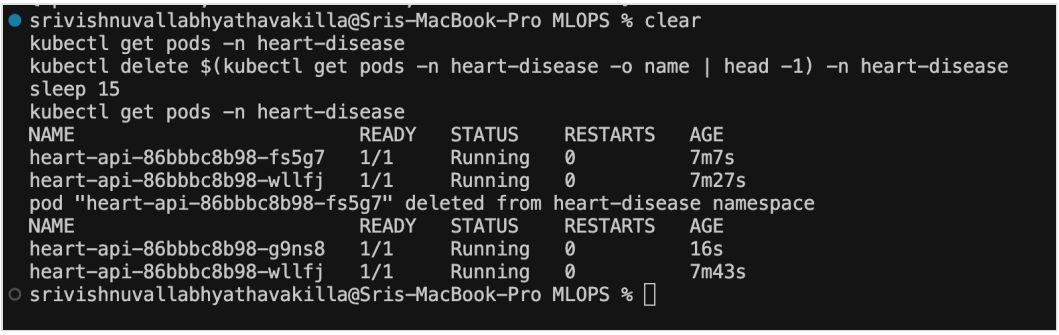


Figure 17 – Self-healing: a deleted pod is replaced by a new one.

Access instructions for local testing

```
docker build -t heart-api:1.1 .
kubectl apply -f k8s/namespace.yaml && kubectl apply -f k8s/ && kubectl apply -f
k8s/monitoring/
curl http://localhost:8080/health
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d
@sample_patient.json
```

API docs: http://localhost:8080/docs. Prometheus: http://localhost:9090. Grafana: http://localhost:3000 (admin / admin) → Dashboards → MLOps → Heart Disease API. Full steps are in the README.

10. Monitoring and logging

- Logging: every request writes one JSON line (path, status, time taken), and every prediction writes another (label, confidence). JSON lines are easy to search and filter.
- Metrics on /metrics: request count (by endpoint and status), response time, predictions by result, and model confidence.
- Prometheus finds the API pods by itself through Kubernetes and reads /metrics every 5 seconds. This matters because pod names change after a restart, as the self-healing test showed.
- Grafana shows an 8-panel dashboard that is loaded automatically.

During a traffic test with random patients (about 10% made invalid on purpose), the dashboard showed: 2 pods up, 540 predictions, 10.4% error rate (the deliberate invalid requests), 82.7% average confidence, a p95 response time of about 24 ms, and requests shared evenly between the two pods.

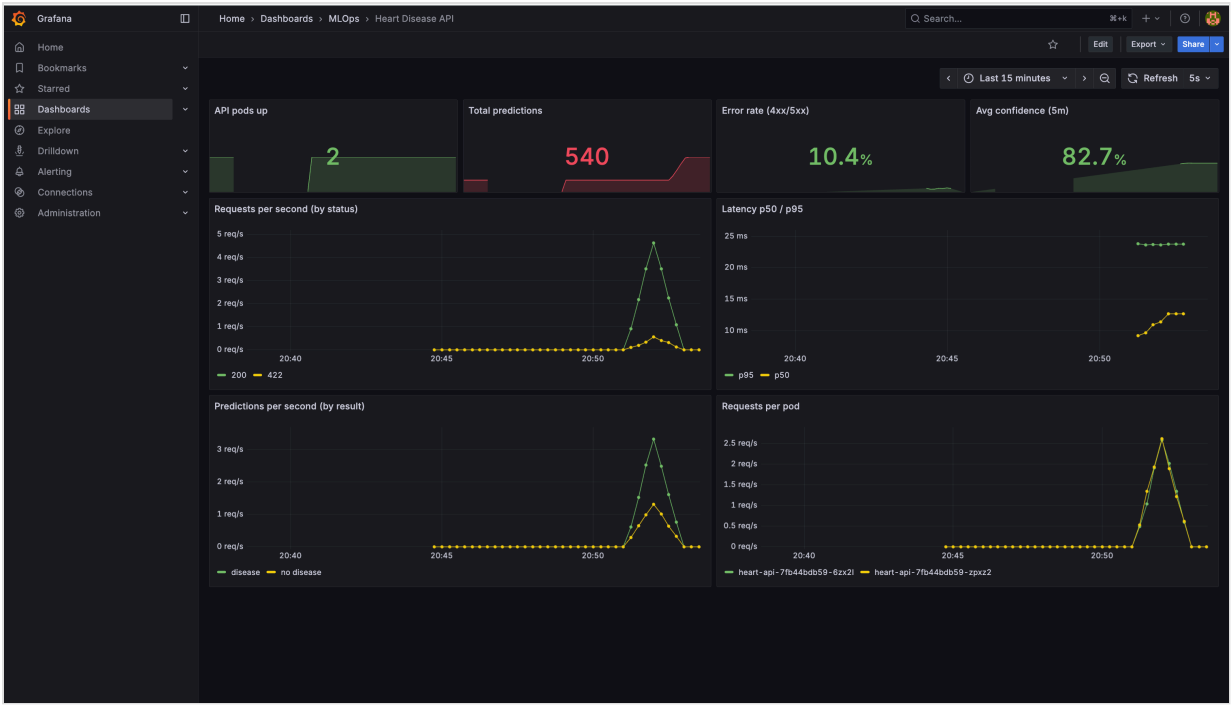


Figure 18 – Grafana dashboard during the traffic test.

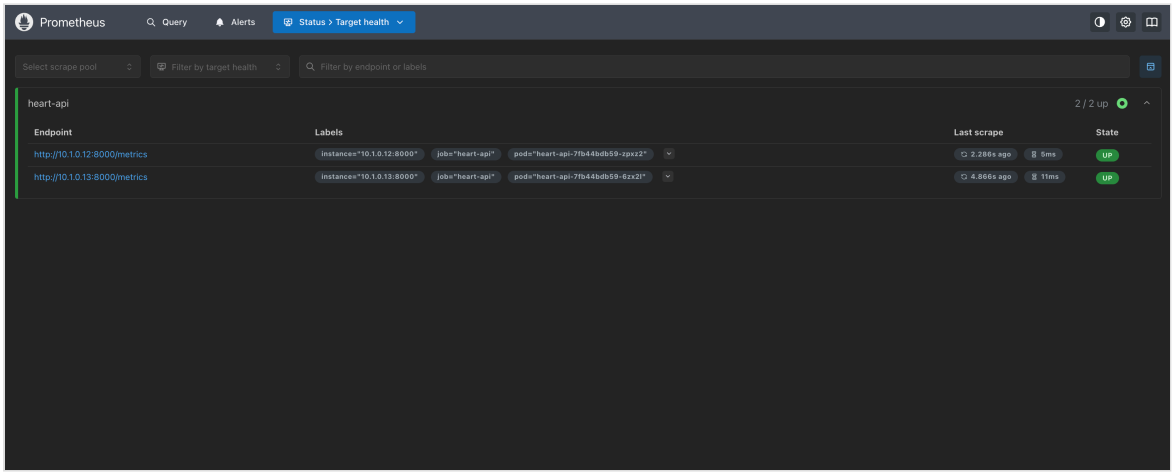


Figure 19 – Prometheus is collecting metrics from both pods.



Figure 20 – JSON request and prediction logs from a pod.

11. Conclusion and next steps

The project works end to end: the data can be downloaded and cleaned with scripts, experiments are tracked, the model and its preprocessing are saved together, every push is tested automatically, and the model runs as a monitored, self-healing service on Kubernetes.

Limitations: the dataset is small (303 patients), so small score differences are not meaningful; the cluster is local; and the image is not stored in a registry.

Next steps: deploy to a cloud Kubernetes service with an image registry, add alerts when the error rate or response time is too high, watch the prediction mix and confidence over time to detect data drift, and use the MLflow Model Registry for model versions.

Links

- Code repository: https://github.com/Vishnuvallabh/heart-disease-mlops
- Video: [video link]